

Detection of AI vs Human generated text using PyTorch models.

Contributors	Roll nos	Project Roles
Aditi Govindu da25g501	DA25G501	Data and ML Engineer
Arvind M Raja da25g505	DA25G505	DevOps Engineer
Lavanya Srinivasan Ramadurai da25g516	DA25G516	Quality Assurance and Testing
G K Sreevineeth da25m623	DA25M623	System design and development

Creation date: Jul 19, 2026

Last updated date: Aug 21, 2026

Problem Statement

The advent of Generative AI (GenAI) has led to a large corpus of fake news articles. The news is unbiased and states the facts, but the scale, volume and near-real composition of text makes it difficult to validate the source of the news - Human or AI. In a world advancing towards Artificial General Intelligence (AGI), this project empowers humans to identify if content was AI generated or not. It allows users to fact check information and reduce AI slop in their lives.

Our team's goal is to compare and contrast fact based human words to overtly expressive language used by AI with lexical word matching.

Dataset description

The [dataset](#) used for this project comes from Kaggle. We have used 1500 rows of text with the following attributes:

- id - Unique identifier for every row in the dataset from 1 to 1500.
- label - Target decision value to be learnt by the model as "AI-generated" or "Human-written"
- prompt - Title of the news article
- model - Name of the source of the data. It can be human or AI generated tools like Claude, GPT-4, Gemini, ChatGPT etc.
- date - The date the news article was published

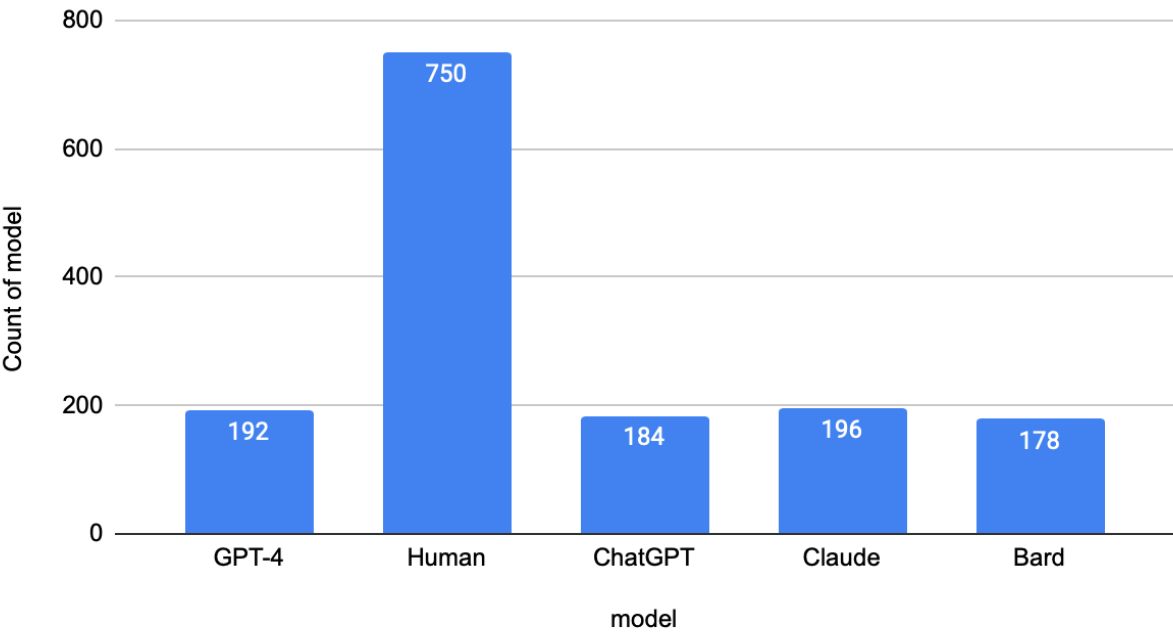
The dataset is perfectly balanced with 750 samples for AI and human generated content each, as depicted in diagram 1 below.

Human vs AI labels



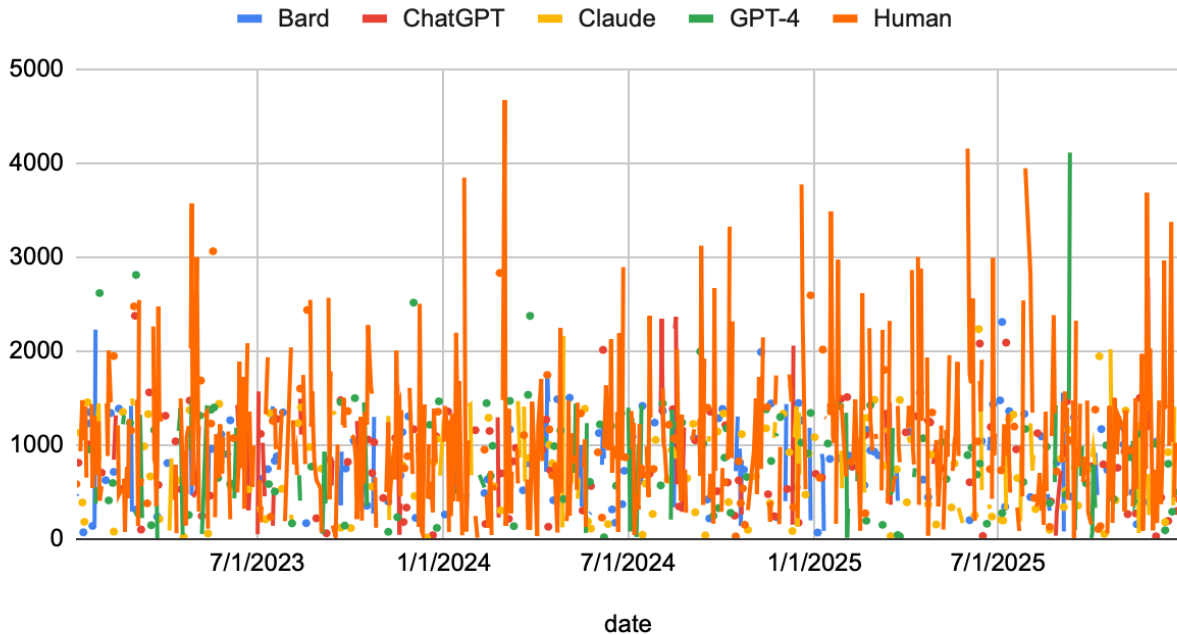
Diagram 2 depicts data stratified per source of label, with AI generated labels split across GPT-4 (192), ChatGPT (184), Claude (196) and Gemini / Bard (178).

Count of samples per article title



Data is collected from news articles over every 6 months from 2023 to 2025. Data spikes are seen for new model releases starting from 1-1-2023 to 1-7-2025, as illustrated in figure 3.

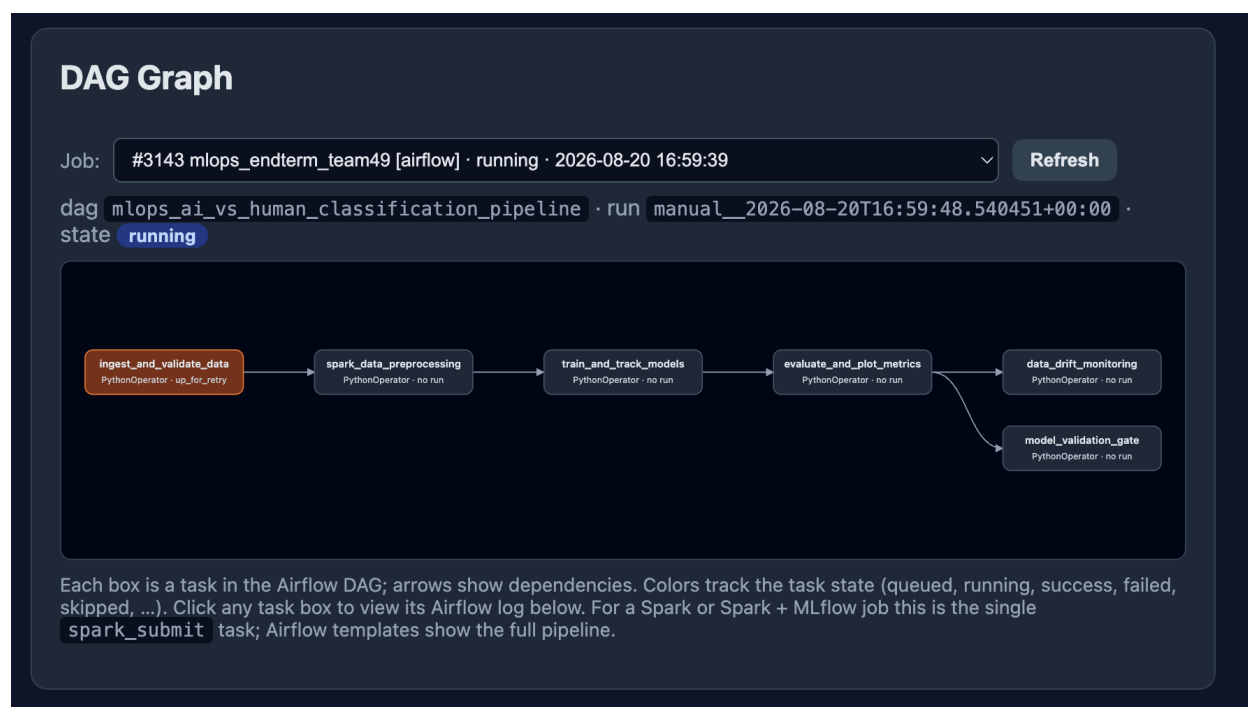
Bard, ChatGPT, Claude, GPT-4 and Human



System Architecture

The human vs AI pipeline is designed as depicted in [System Architecture](#) on the project GitHub page. The [airflow DAG](#) is illustrated in figure 4. There are 5 broad stages:

- Data processing using Apache Spark. This includes ingestion, preprocessing, creation and storage of parquet files for training of the model.
- Model training with traditional ML algorithms and PyTorch deep learning model. The traditional approach uses TF-IDF vectorizer with Logistic Regression, Random Forest and Naive Bayes in the training phase. The TF-IDF model with logistic regression outperformed the other models, hence is pushed as a baseline model to the next phase. For deep learning, the PyTorch model BiLSTM with self attention (128 hidden nodes) is used to train the model.
- Deployment phase comprises creation of DockerFile for each of these models.
- The models are hosted using FastAPI for real time serving of user requests.
- Monitoring and maintenance includes definition of Prometheus YAML files and definition of endpoints for /metrics in the same API as the FastAPI.



These stages are algorithmically expressed in the following section.

flowchart TD

```

    subgraph Data_Engineering ['Data Ingestion & Engineering']
        A[Raw Dataset: ai_vs_human_text.csv] --> B[Schema Validation & Cleaning]
        B --> C[Apache Spark PySpark Pipeline]
        C --> D[Distributed Tokenisation & Feature Extraction]
        D --> E[DVC Data Versioning]
    end

    subgraph Orchestration ['Workflow Orchestration']
        AF[Apache Airflow DAG]
        AF -->|Triggers| B
        AF -->|Triggers| C
        AF -->|Triggers| F[Baseline Model: TF-IDF + Logistic Regression]
        AF -->|Triggers| J[Evaluation Suite]
        AF -->|Triggers| N[Statistical Drift Detector: KS-Test & PSI]
    end

    subgraph Model_Development ['Model Development & Tracking']
        D --> F
        D --> G[Deep Learning: PyTorch BiLSTM + Attention]
        F & G --> H[MLflow Experiment Tracking & Registry]
    end
  
```

```

        H -->|Logs Metrics, Parameters, Checkpoints| I[(MLFlow
Tracking Server)]
    end

    subgraph Evaluation ['Model Evaluation & Quality Gate']
        G --> J[Evaluation Suite]
        J --> K[Confusion Matrices & ROC Curves]
        J --> L[Quality Gate Check F1 >= 0.70]
    end

    subgraph Deployment ['Deployment & Serving']
        L -->|Certified Model| M[Dockerized FastAPI Application]
        M -->|REST API| P[POST /predict & /batch-predict]
        M -->|Exposes Metrics| Q[GET /metrics]
    end

    subgraph Monitoring ['Monitoring & Drift Detection']
        P --> N[Statistical Drift Detector: KS-Test & PSI]
        N --> O[Drift Reports: JSON & HTML]
        Q --> R[Prometheus Server]
        R --> S[Grafana Dashboards]
    end
end

```

Data Pipeline

The data pipeline is defined in the AirFlow [DAG module](#) with 6 stages

1. Ingest Data (Schema validation & cleaning)
2. Spark Preprocessing (PySpark distributed feature engineering & partitioning)
3. Model Training (Baseline + PyTorch BiLSTM with MLflow tracking)
4. Model Evaluation (Confusion matrix & ROC curve generation)
5. Data Drift Monitoring (KS-test & statistical drift reporting)
6. Model Validation & Deployment Gate

Of these, Ingest Data and Spark Preprocessing belong to the data pipeline stages. The data pipeline comprises the following steps:

- [Ingestion](#) of data from a csv format.
- Schema of the file is validated to ensure id, text and labels are present.
- [Preprocessing](#) of data includes removal of missing values.
- All text is converted to lowercase, URLs and HTML tags are removed, irregular punctuations or emojis are purged and text is stripped of leading or trailing white spaces.
- Lexical features of the dataset that quantify its performance are collected such as character length, word count, average word length, type-token ratio etc.

- The dataset is tested for balance of labels in human-written vs AI-generated labels and pushed ahead once validated. It is augmented with fake samples of the smaller class, if disbalanced.
- The dataset is split into 70% training, 15% validation and 15% testing sets and saved as parquet files.
- All operations are bundled into a single Apache Spark operation, which can handle multiple rows of data and parallel processing.

Model development

The baseline model serves as a benchmark to compare the PyTorch deep learning model. In this experiment, the baseline [model](#) uses TF-IDF vectorizer with Logistic Regression, Random Forest and Multinomial Naive Bayes. The precision, recall and F1 scores of these models are compared to finalize the baseline model.

Term Frequency - Inverse Document Frequency (TF-IDF)

Term frequency Inverse Document Frequency is a numerical feature extraction technique to convert raw text into rational vector numbers. It measures 2 metrics:

- Term Frequency which counts the number of times a word appears in the text.
- Inverse Document Frequency computes the inverse of TF from stage 1 to penalize repetitive stops words and reward rare words.

Multinomial Naive Bayes (MNB)

MNB is a probabilistic generative classifier based on Bayes' Theorem, that naively assumes features are independent in a sentence. The posterior probability of the next word is obtained by applying Bayes theorem to previously seen words. This suffers in text based challenges, especially news articles where every sentence depends on the surrounding context. This is unable to capture sentence meanings or emotional nuance and is not taken forward for baseline.

Random Forest (RF)

This is an ensemble method built by training multiple child decision trees. Bootstrap aggregation or bagging is applied to the decision of multiple sub-trees to achieve a final consensus verdict. Every sub-tree is trained on a subset of the features and words in the text. The dataset is sparse with 4 features and 1500 samples. This leads to sparse matrix representation of the dataset, which is not enough to train an RF model. To avoid overfit, we avoid considering this model as the baseline.

Logistic Regression (LR)

A linear classification algorithm that models the probability of a binary outcome. It takes the weighted sum of the input features from TF-IDF and passes them to a linear equation to estimate the y-intercept and slope. Since the problem statement requests classification, the logistic regression model output is passed through a sigmoid function that compresses values to 0 (Human-written) or 1 (AI-generated). It uses the binary cross entropy or log loss function for loss computation.

TF-IDF with logistic regression is selected as the baseline model since it runs in a few seconds, it is the simplest model with minimal memory requirements. The linear coefficients can be used to easily represent the labels for a binary classification (AI-generated vs Human-written) due to a well calibrated sigmoid function.

Bidirectional Long Short Term Memory (BiLSTM)

The PyTorch deep learning model comprising the BiLSTM model with self attention offers semantic comprehension and context identification. Traditional Natural Language Processing (NLP) models, such as the baseline TF-IDF with Logistic Regression, considers text as an unordered “Bag of Words”.

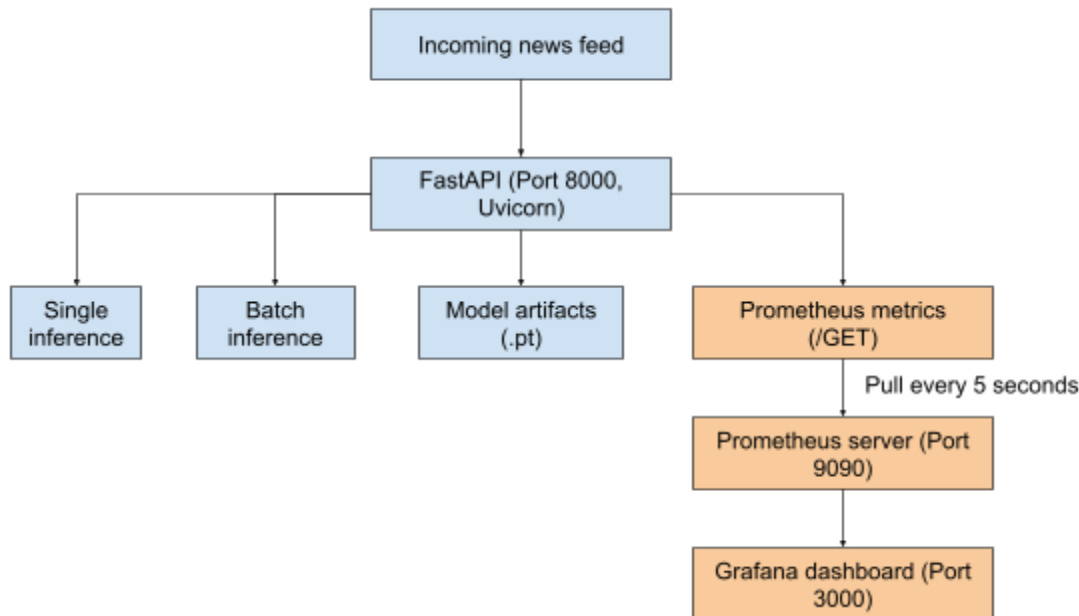
However, AI-generated text differs from authentic human journalism not just by individual keywords, but by structural syntax, sentence rhythm, clause transitions, and long-range semantic coherence. To detect these subtle patterns, the model must understand word order and sequential context. It must understand and represent the semantic context of the input information.

The BiLSTM model consists of a forward pass and backward pass of tokens in a string. Unlike traditional Recurrent Neural Networks (RNNs) that forget older tokens, the LSTM model introduces an input, forget and output gate to hold up to 256 tokens at a point of time. The forward pass identifies previous tokens, while the backward pass identifies the context in which previous tokens were used. This enables BiLSTM to build a contextual representation. In a standard LSTM, only the hidden state of the final token or a simple average is passed to the classifier. This creates an information bottleneck and suffers from recency bias. Self attention provides a weight to each input token, thus eliminating the recency bias. Sufficient training enables the model to provide journalistic attribution markers used by humans with a higher weight (E.g. "said", "interview", "pointed out"). It also highlights synthetic transition markers, such as "furthermore", "important to note", etc.

Since news articles are factual with limited jargon by humans, as opposed to AI generated news articles, BiLSTM is sufficient compared to Bidirectional Encoder Representations from Transformers (BERT) which requires GPU and large memory for training. BiLSTM layers can be defined and used directly in Pytorch, instead of calling BERT from the HuggingFace API, making this deployment secure and airtight for deployment on secure servers.

Deployment strategy

This project is deployed through a microservices architecture. This is depicted in the figure 5 below.



Before deployment to the FastAPI endpoint, a quality check is performed on the PyTorch model to ensure F1 score > 70% and accuracy > 90%. Accuracy is held to a higher accountable threshold, given balanced dataset training. Once the PyTorch weights are loaded in a pt file and vocabularies are identified with no loss of data, the model is deployed using DockerFile.

The Docker image loads `python 3.9 slim` for fast deployment. `Requirements.txt` is installed first to take advantage of docker layer caching and `--no-cache-dir` is applied to ensure cache for local pip installs is always cleared to reduce conflicts. `RUN mkdir -p config src data models reports logs` ensures zero runtime missing-folder errors and Docker container health is checked before deployment on the FastAPI endpoint.

With lifespan context, the BiLSTM model weights and vocabulary are loaded into memory on application startup. This eliminates disk I/O latency during inference requests, enabling sub-15ms response times. If the PyTorch model is unavailable, a fallback to the baseline TF-IDF + LR model is enabled, to reduce poor user experience. The exposed endpoints of the FastAPI are:

- POST /predict: Low latency real time classification of single articles.
- POST /predict: Low latency real time classification of a group of articles.
- GET /health: Probe for docker status and load balancers
- GET /metrics: Native prometheus exposed endpoint for metrics to be read.

- POST /drift-check: On demand statistical drift evaluation against the training baseline.

To reduce redundant command runs, a docker-compose.yml file is established to orchestrate the MLOPs pipeline.

Table 1 below summarises the container services, port numbers and purpose in deployment.

Container service	Image / Port number on user machine	Role in deployment
api	Custom DockerFile / 8000	Serves inference requests and hosts metrics endpoints for Prometheus.
mlflow	ghcr.io/mlflow/mlflow / 5000	Model artifacts and experiment parameter values.
prometheus	prometheus / 9090	Pulls latency and gauge metrics every 5 seconds to expose at metrics endpoint.
grafana	grafana/grafana / 3000	Visualize throughput, P95 latency and prediction distribution of the models.
airflow	apache/airflow:2.7.1 / 8080	Orchestrates automated retraining of a directed acyclic graph (DAG) on a schedule.

Monitoring strategy

Model's performance and drift rate after deployment is measured to ensure consistency in product offerings to the end user. It helps identify context poisoning and alerts developers to take immediate corrective action. Monitoring of the PyTorch model is divided into 2 categories:

Operational monitoring

Operational monitoring focuses on latency and end user experience. It ensures the model meets the Service Level Agreements and Objectives (SLAs / SLOs) defined at the time of ideation. Prometheus is used to measure the real time performance of the model with counter and gauge metrics, such as requests count, response time, error counts (4xx / 5xx), latency in P99 services etc. Latency is measured every 5 seconds and a deviation of 3% for more than 2 minutes indicates triage is needed.

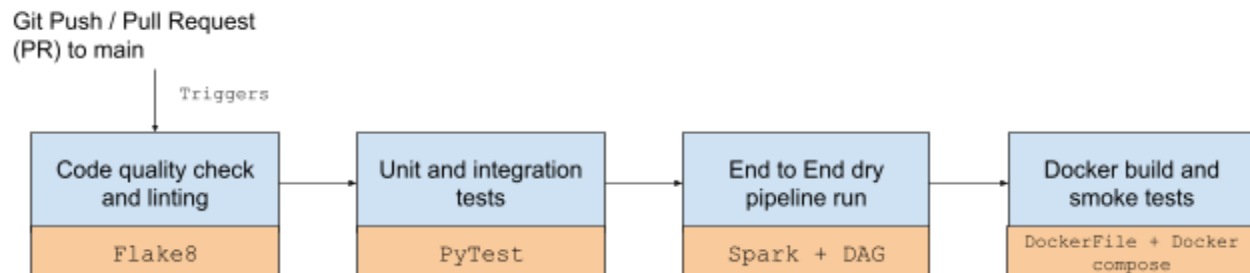
Statistical quality monitoring

Statistical data drift is used to measure differences in tests on incoming article features, such as type-token ratio (TTR), word count, character length and other lexical features. A deviation of 10% or a p-value

above 0.05 in Kolmogorov-Smirnov (KS) tests indicates that the drift is statistically significant and must be triaged before exposure to the end user. Population Stability Index (PSI) on prediction confidence is also used to identify if language style drifts are seen, triggering an automated retraining request.

CI/CD implementation

CI/CD automation is controlled via GitHub Actions in a public repository. These actions are orchestrated by the `.github/workflows/ci-cd.yml` file. Every git push or pull_request to main triggers a 4-phase automated CI/CD pipeline illustrated in figure 6.

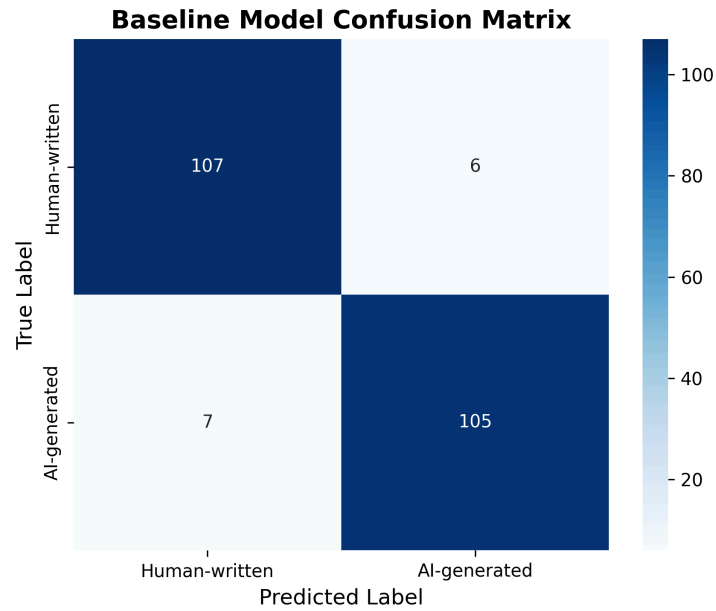


Results & Discussion

Results comparing the performance of the Baseline TF-IDF and Logistic Regression model to the PyTorch model are depicted in table 1 below. Both models are evaluated on 225 texts each. PyTorch outperformed the baseline model by ~3% precision and ~1% recall points.

Metric	Baseline (TF-IDF + LR)	PyTorch (BiLSTM + Attention)
Precision	91.59%	94.59%
Recall	92.75%	93.75%
F1-Score	91.17%	94.17%
Accuracy	92.22%	94.22%
ROC-AUC	93.89%	93.96%

The confusion matrix for the baseline model can be observed in diagram 7.

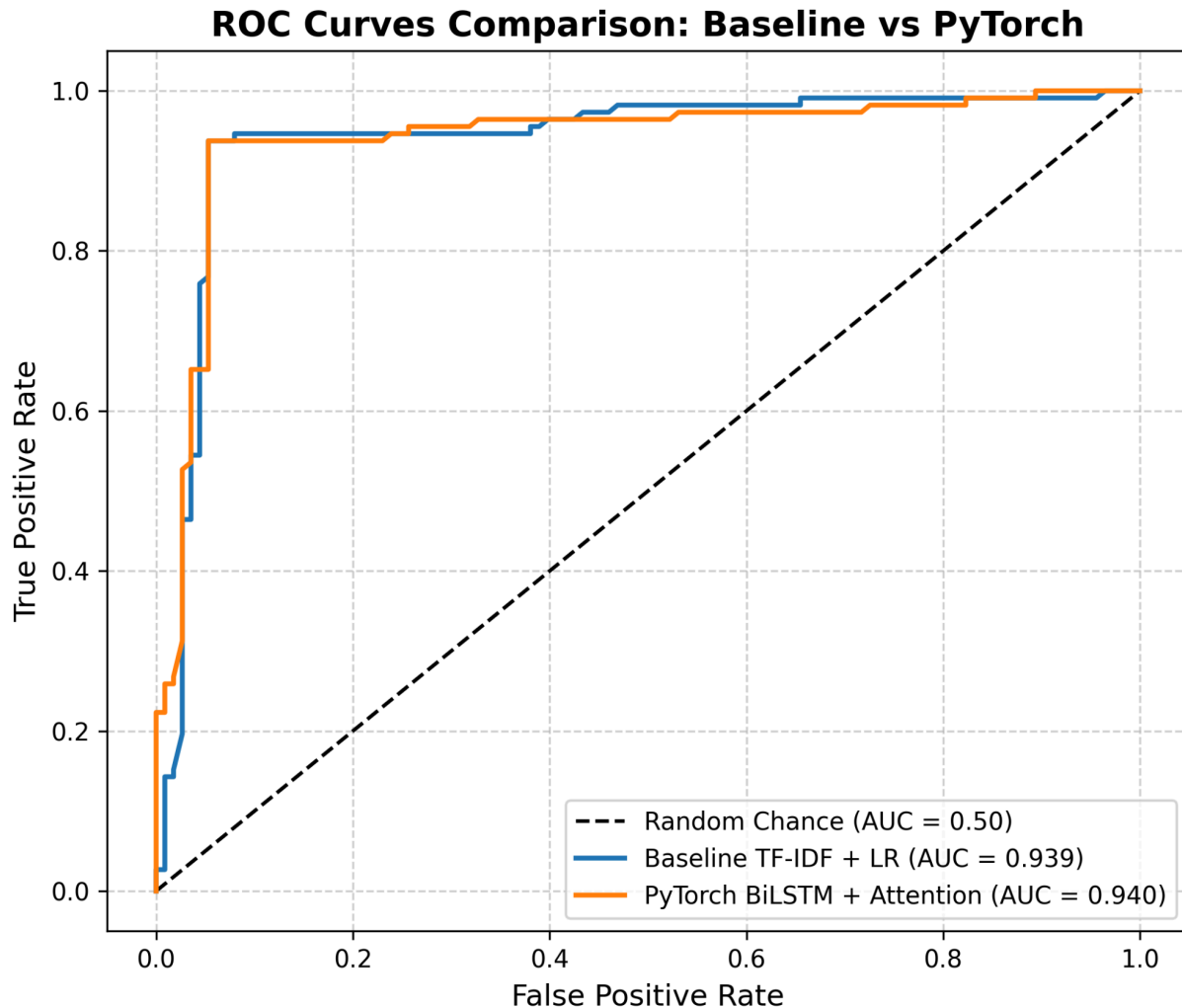


The following table denotes the confusion matrix of labels predicted by the PyTorch model, where:

- True Positives (TP) = 107
- True Negatives (TN) = 105
- False Positives (FP) = 6
- False Negatives (FN) = 7

	Predicted Human	Predicted AI
True Human	107	6
True AI	7	105

Diagram 8 illustrates the difference in Receiver Operating characteristics or True Positive Rate (TPR) and False Positive Rate (FPR) of both the models in an ROC-AUC curve.



Challenges faced

The team faced challenges in:

- Collection of adequate data and samples from a large number of news articles. Given the recent increase in AI models, a sizable amount of time was spent on manually vetting the labels and increasing the sample size to reach 1500 cases.

Future improvements

The current model is designed to support text modality and we firmly believe that this can be extended to image, video and audio modalities to become a cross functional tool for social media channels.

Bibliography

- https://github.com/aditi-govindu/DA5402W_project_49_da25g501
-  [mlops-project-team49-artifacts](#)
-  [Notes - MLOps Project discussion](#)
- <http://localhost:5000/#/experiments/1?viewStateShareKey=3894e7dac091113a949e1a0b144bdfbf23f857b1cfb2b6251e919052fe25b155&compareRunsMode=TABLE>
- <https://airflow.apache.org/docs/>
- <https://docs.docker.com/>
- <https://www.sciencedirect.com/topics/computer-science/bidirectional-long-short-term-memory-network>
- <https://arxiv.org/html/2605.07793v1>
- <https://docs.github.com/en/billing/concepts/product-billing/github-actions#how-use-of-github-actions-is-measured>
- <https://docs.github.com/en>